

**ADMM: примеры, солверы. Непрерывные
методы оптимизации.**

Даня Меркулов

ФКН ВШЭ

ADMM

Сюжет лекции

Лекция состоит из двух частей.

Сюжет лекции

Лекция состоит из двух частей.

- **Часть I — ADMM** (продолжение лекции 19). Метод, соединяющий надёжность модифицированной функции Лагранжа с декомпозицией двойственного подъёма. Идея, шаги, невязки, консенсус, применения и реальные солверы.

Сюжет лекции

Лекция состоит из двух частей.

- **Часть I — ADMM** (продолжение лекции 19). Метод, соединяющий надёжность модифицированной функции Лагранжа с декомпозицией двойственного подъёма. Идея, шаги, невязки, консенсус, применения и реальные солверы.

Сюжет лекции

Лекция состоит из двух частей.

- **Часть I — ADMM** (продолжение лекции 19). Метод, соединяющий надёжность модифицированной функции Лагранжа с декомпозицией двойственного подъёма. Идея, шаги, невязки, консенсус, применения и реальные солверы.
- **Часть II — непрерывные методы.** Взгляд на оптимизацию как на траекторию обыкновенного дифференциального уравнения: градиентный поток и ускоренный поток, с доказательствами скоростей сходимости.

Сюжет лекции

Лекция состоит из двух частей.

- **Часть I — ADMM** (продолжение лекции 19). Метод, соединяющий надёжность модифицированной функции Лагранжа с декомпозицией двойственного подъёма. Идея, шаги, невязки, консенсус, применения и реальные солверы.
- **Часть II — непрерывные методы.** Взгляд на оптимизацию как на траекторию обыкновенного дифференциального уравнения: градиентный поток и ускоренный поток, с доказательствами скоростей сходимости.

Сюжет лекции

Лекция состоит из двух частей.

- **Часть I — ADMM** (продолжение лекции 19). Метод, соединяющий надёжность модифицированной функции Лагранжа с декомпозицией двойственного подъёма. Идея, шаги, невязки, консенсус, применения и реальные солверы.
- **Часть II — непрерывные методы.** Взгляд на оптимизацию как на траекторию обыкновенного дифференциального уравнения: градиентный поток и ускоренный поток, с доказательствами скоростей сходимости.

Начнём ровно с того места, где остановилась прошлая лекция, — с ADMM.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный
подъём

- **Разбивает** задачу на блоки
(параллельно по x_i).

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.
- Расщепляет гладкое и негладкое через x, z, ν .

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- **Разбивает** задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.
- Расщепляет гладкое и негладкое через x, z, ν .

ADMM: соединение двух подходов

ADMM — *alternating direction method of multipliers*. Название не переводим, дальше используем только аббревиатуру.

Мы разобрали два метода с противоположными достоинствами:

Двойственный градиентный подъём

- Разбивает задачу на блоки (параллельно по x_i).
- Требуется сильной выпуклости, сходится медленно.

Метод модифицированной функции Лагранжа (ALM)

- **Надёжен**: сходится при слабых условиях.
- Штраф $\|Ax - b\|^2$ связывает переменные, поэтому **теряем декомпозицию**.

ADMM

- Надёжность ALM (сходится при слабых условиях).
- Декомпозиция: x и z обновляются **по очереди**.
- Расщепляет гладкое и негладкое через x, z, ν .

Основная идея: вводим разделяющую переменную ($Ax + Bz = c$), чтобы f и g минимизировать **по отдельности**, и связываем их штрафом и множителем.

ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь $r(z)$ — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция $g(\nu)$ из прошлых разделов.

ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь $r(z)$ — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция $g(\nu)$ из прошлых разделов.

Добавим к целевой функции штраф за нарушение ограничения:

$$\min_{x,z} f(x) + r(z) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

$$\text{при } Ax + Bz = c$$

ADMM: общая постановка

Задача оптимизации:

$$\min_{x,z} f(x) + r(z)$$

$$\text{при } Ax + Bz = c$$

Здесь $r(z)$ — второе слагаемое целевой функции (обычно негладкое), а **не** двойственная функция $g(\nu)$ из прошлых разделов.

Добавим к целевой функции штраф за нарушение ограничения:

$$\min_{x,z} f(x) + r(z) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

$$\text{при } Ax + Bz = c$$

где $\rho > 0$ — параметр. Модифицированная функция Лагранжа:

$$L_\rho(x, z, \nu) = f(x) + r(z) + \nu^T (Ax + Bz - c) + \frac{\rho}{2} \|Ax + Bz - c\|^2$$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь ν — множитель ограничения, шаг по нему идёт с коэффициентом ρ .

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь ν — множитель ограничения, шаг по нему идёт с коэффициентом ρ .

Отличие от ALM. Обычный ALM заменил бы первые два шага совместной минимизацией:

$$(x_k, z_k) = \arg \min_{x, z} L_\rho(x, z, \nu_{k-1})$$

Шаги ADMM (формально)

ADMM повторяет следующие шаги для $k = 1, 2, 3, \dots$:

1. Обновить x : $x_k = \arg \min_x L_\rho(x, z_{k-1}, \nu_{k-1})$
2. Обновить z : $z_k = \arg \min_z L_\rho(x_k, z, \nu_{k-1})$
3. Обновить ν : $\nu_k = \nu_{k-1} + \rho(Ax_k + Bz_k - c)$

здесь ν — множитель ограничения, шаг по нему идёт с коэффициентом ρ .

Отличие от ALM. Обычный ALM заменил бы первые два шага совместной минимизацией:

$$(x_k, z_k) = \arg \min_{x, z} L_\rho(x, z, \nu_{k-1})$$

ADMM **не** минимизирует (x, z) совместно, иначе переменные снова оказались бы связаны. Один проход по каждой переменной обходится чуть большим числом итераций, зато подзадачи f и r решаются по отдельности, часто в замкнутой форме.

Масштабированная форма: компактные шаги

Введём масштабированный множитель $u = \nu/\rho$: он прячет ρ внутрь нормы. Завершая квадрат, $\nu^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|^2 = \frac{\rho}{2}\|Ax + Bz - c + u\|^2 - \text{const}$. Получаем компактную запись (исходный множитель $\nu = \rho u$):

$$x_k = \arg \min_x f(x) + \frac{\rho}{2}\|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2}\|Ax_k + Bz - c + u_{k-1}\|^2$$

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Масштабированная форма: компактные шаги

Введём масштабированный множитель $u = \nu/\rho$: он прячет ρ внутрь нормы. Завершая квадрат, $\nu^T(Ax + Bz - c) + \frac{\rho}{2}\|Ax + Bz - c\|^2 = \frac{\rho}{2}\|Ax + Bz - c + u\|^2 - \text{const}$. Получаем компактную запись (исходный множитель $\nu = \rho u$):

$$x_k = \arg \min_x f(x) + \frac{\rho}{2}\|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

$$z_k = \arg \min_z r(z) + \frac{\rho}{2}\|Ax_k + Bz - c + u_{k-1}\|^2$$

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

В этой форме u -шаг идёт без ρ , поскольку её роль уже внутри нормы. Далее используем именно эту форму.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

- ρ играет роль жёсткости связи между x и z : при большом ρ согласование быстрее, но оптимизация каждой части медленнее.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($\nu = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

- ρ играет роль жёсткости связи между x и z : при большом ρ согласование быстрее, но оптимизация каждой части медленнее.

Интуиция: что делают шаги $x \rightarrow z \rightarrow u$?

Шаг x — реши свою половину, держа z фиксированным:

$$x_k = \arg \min_x f(x) + \frac{\rho}{2} \|Ax + Bz_{k-1} - c + u_{k-1}\|^2$$

Шаг z — подстройся под новый x :

$$z_k = \arg \min_z r(z) + \frac{\rho}{2} \|Ax_k + Bz - c + u_{k-1}\|^2$$

Шаг u — накопи рассогласование (интегральная обратная связь):

$$u_k = u_{k-1} + (Ax_k + Bz_k - c)$$

Здесь u — масштабированный множитель ($v = \rho u$), поэтому ρ ушёл внутрь нормы.

Интерпретация. Шаги x и z тянут общее решение каждый к своему оптимуму (x отвечает за гладкую часть, z — за негладкую часть и ограничение). Множитель u растёт ровно настолько, насколько расходятся x и z , и штрафом сближает их.

- ρ играет роль жёсткости связи между x и z : при большом ρ согласование быстрее, но оптимизация каждой части медленнее.
- **Связь с прокс-методом.** При $B = I$ шаг z — это в точности проксимальный оператор: $z_k = \text{prox}_{r/\rho}(Ax_k + u_{k-1})$. ADMM обобщает проксимальный метод: прокс негладкой части r и шаг по гладкой f разнесены на разные переменные.

Невязки ADMM: критерий остановки и настройка ρ

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .
- $\|s_k\| \gg \|r_k\|$: решение слишком инертно $\Rightarrow$ **уменьшить ρ** .

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .
- $\|s_k\| \gg \|r_k\|$: решение слишком инертно $\Rightarrow$ **уменьшить ρ** .

Невязки ADMM: критерий остановки и настройка ρ

Прямая невязка измеряет нарушение ограничения (рассогласование x и z):

$$r_k = Ax_k + Bz_k - c$$

Двойственная невязка измеряет, насколько ещё меняется решение между итерациями:

$$s_k = \rho A^T B(z_k - z_{k-1})$$

В ККТ-оптимуме **оба равны нулю**. Критерий остановки:

$$\|r_k\| \leq \varepsilon^{\text{pri}}, \quad \|s_k\| \leq \varepsilon^{\text{dual}}.$$

Баланс ρ :

- $\|r_k\| \gg \|s_k\|$: ограничение нарушено сильнее $\Rightarrow$ **увеличить ρ** .
- $\|s_k\| \gg \|r_k\|$: решение слишком инертно $\Rightarrow$ **уменьшить ρ** .

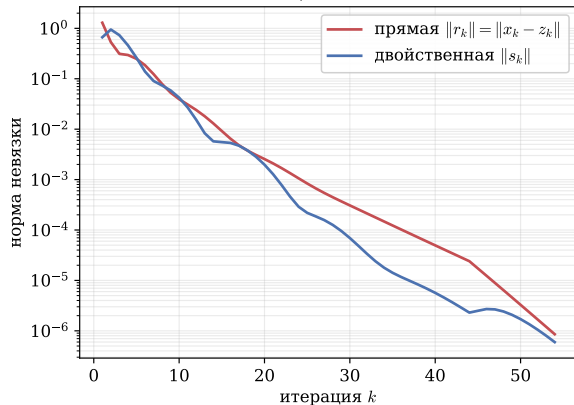
Адаптивное правило (Boyd et al., §3.4.1):

$$\rho_{k+1} = \begin{cases} 2\rho_k, & \|r_k\| > 10\|s_k\| \\ \rho_k/2, & \|s_k\| > 10\|r_k\| \\ \rho_k, & \text{иначе} \end{cases}$$

Простая эвристика, которая часто заметно ускоряет сходимость.

Невязки и адаптивный ρ : эксперимент

Адаптивный ρ : обе невязки $\rightarrow 0$



Цена неудачного ρ против адаптивного

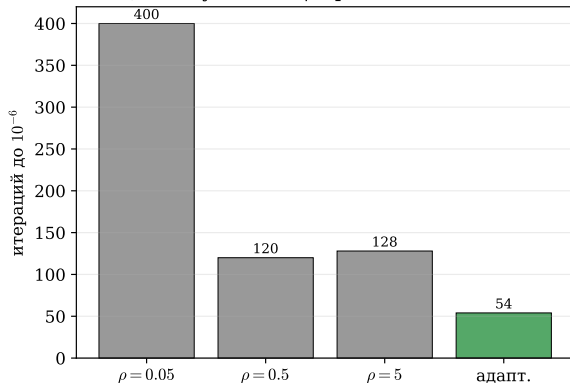


Figure 1: Слева: прямая и двойственная невязки ADMM на LASSO с адаптивным ρ , обе монотонно падают до 10^{-6} . Справа: число итераций до этой точности; неудачно выбранное фиксированное $\rho = 0.05$ заметно медленнее, а правило балансировки невязок сходится быстрее всех протестированных фиксированных ρ без ручного подбора (с одной рефакторизацией).

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.
- **Глобально (усреднение):** z есть **усреднение** локальных решений (и их множителей).

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.
- **Глобально (усреднение):** z есть **усреднение** локальных решений (и их множителей).

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Консенсусный ADMM: оптимизация на нескольких машинах

Задача: минимизировать сумму, в которой каждое f_i хранится на своей машине (свой кусок данных):

$$\min_x \sum_{i=1}^N f_i(x).$$

Вводим **локальные копии** x_i и общую консенсус-переменную z :

$$\min_{x_1, \dots, x_N, z} \sum_{i=1}^N f_i(x_i) \quad \text{при} \quad x_i = z, \quad i = 1, \dots, N.$$

Итерации ADMM становятся **полностью параллельными**:

$$x_i^k = \arg \min_{x_i} f_i(x_i) + \frac{\rho}{2} \|x_i - z^{k-1} + u_i^{k-1}\|^2$$

$$z^k = \frac{1}{N} \sum_{i=1}^N (x_i^k + u_i^{k-1})$$

$$u_i^k = u_i^{k-1} + x_i^k - z^k$$

Структура связи: локально $\rightarrow$ глобально

- **Локально (параллельно):** каждая машина решает свою задачу x_i по своим данным.
- **Глобально (усреднение):** z есть **усреднение** локальных решений (и их множителей).

При $\sum_i u_i^0 = 0$ имеем $z^k = \frac{1}{N} \sum_i x_i^k$.

Так ADMM применяют в федеративном обучении: данные не покидают узел, обмен идёт только векторами.

Консенсусный ADMM в действии

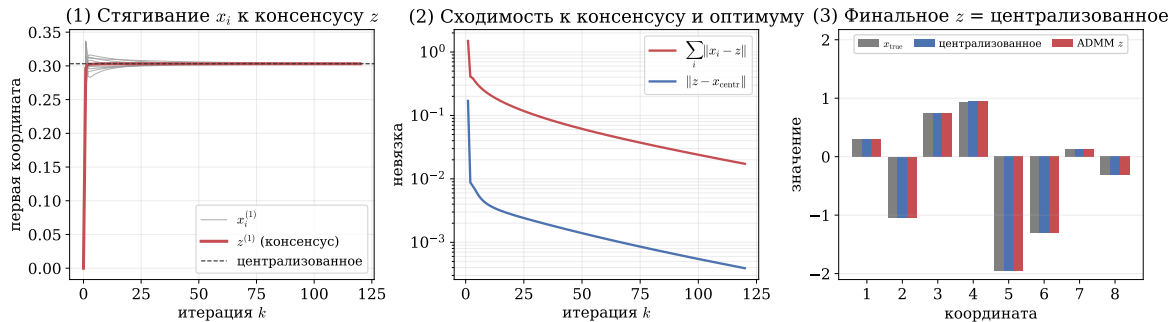


Figure 2: Распределённый МНК, $N = 10$ узлов. Слева: первые координаты локальных x_i (серые) стягиваются к консенсусу z (красная) и к централизованному решению. В центре: расхождение по координатам $\sum_i \|x_i - z\|$ и отклонение $\|z - x_{\text{centr}}\|$ от централизованного решения затухают. Справа: итоговое z совпадает с централизованным; по сети ходят только векторы.

Применения ADMM: один общий приём

За всеми применениями стоит один приём: негладкую часть r отдаём z -шагу, где prox_r имеет **замкнутую форму**; гладкую часть оставляем x -шагу.

Применения ADMM: один общий приём

За всеми применениями стоит один приём: негладкую часть r отдаём z -шагу, где prox_r имеет **замкнутую форму**; гладкую часть оставляем x -шагу.

$r(z)$	z -шаг (prox_r)	где применяется
$\lambda \ z\ _1$	мягкий порог	LASSO, разреженная регрессия
TV-штраф	покоординатное сжатие	TV-восстановление изображений
$\ Z\ _*$	усечение сингулярных чисел	robust PCA (низкий ранг)
$I_{\mathcal{C}}$	проекция на $\mathcal{C}$	конусные ограничения

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

Применения ADMM: где он нужен

- Разреженная регрессия: ℓ_1 , group LASSO.
- TV-обработка изображений: точная негладкая задача (см. далее).
- Распределённое и федеративное обучение: консенсусный ADMM, данные остаются на узлах.
- Низкий ранг плюс разреженность (robust PCA): методы первого порядка здесь не разбивают задачу.

Оговорка. На простом LASSO к высокой точности FISTA сопоставим или быстрее. ADMM выигрывает там, где есть несколько негладких частей, оператор внутри регуляризатора, жёсткие ограничения или распределённость (fused LASSO, TV, консенсус, robust PCA).

ADMM на практике: восстановление смазанного изображения

Смазано + шум (25.9 дБ)



ADMM (28.5 дБ)

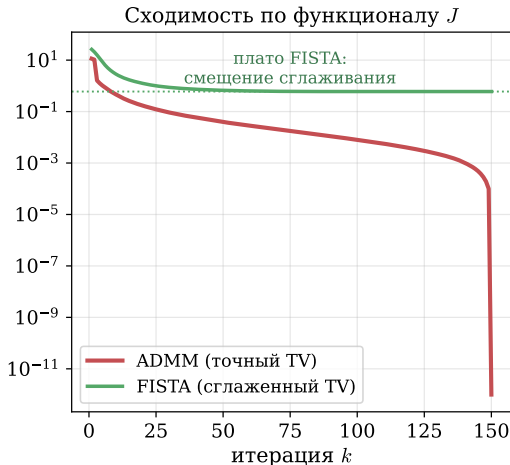


Figure 3: Устранение размытия настоящего фото. ADMM решает **точную** негладкую TV-задачу (FISTA минимизирует **сглаженный** суррогат и смещён по J). По качеству (PSNR) методы сопоставимы: FISTA 28.7 дБ, ADMM 28.5 дБ; ADMM ценен структурой, а не скоростью.

ADMM на практике: восстановление смазанного изображения

Смазано + шум (25.9 дБ)



ADMM (28.5 дБ)

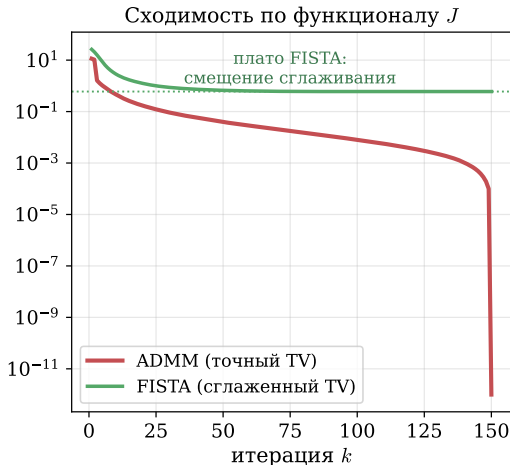


Figure 3: Устранение размытия настоящего фото. ADMM решает **точную** негладкую TV-задачу (FISTA минимизирует **сглаженный** суррогат и смещён по J). По качеству (PSNR) методы сопоставимы: FISTA 28.7 дБ, ADMM 28.5 дБ; ADMM ценен структурой, а не скоростью.

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 4: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 4: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 4: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);
- u -шаг: обновление множителя.

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



Figure 4: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);
- u -шаг: обновление множителя.

ADMM как метод по умолчанию: robust PCA

Robust PCA через ADMM: $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$

Кадр M (фон+объект)



Низкоранг L (фон)



Разреж $|S|$ (движение)



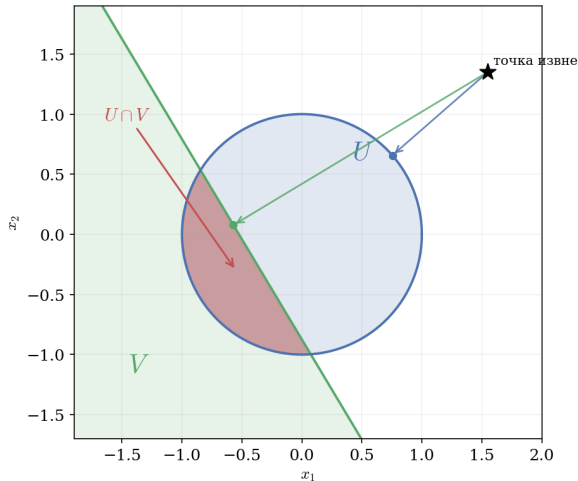
Шаги ADMM в замкнутой форме:

- L -шаг: пороговая обработка сингулярных чисел (прокс ядерной нормы);
- S -шаг: мягкий порог (прокс ℓ_1);
- u -шаг: обновление множителя.

Методы первого порядка не разбивают «ядерная норма + ℓ_1 » так естественно; здесь ADMM есть стандарт.

Figure 4: Вычитание фона: кадры видео суть столбцы матрицы M . ADMM решает $\min \|L\|_* + \lambda \|S\|_1$ при $L + S = M$: L (низкий ранг) — статичный фон, S (разреженная компонента) — движущийся объект.

Пример: метод альтернирующих проекций



Поиск точки в пересечении выпуклых множеств $U, V \subseteq \mathbb{R}^n$:

$$\min_x I_U(x) + I_V(x)$$

Приводим к форме ADMM ($A = I, B = -I, c = 0$):

$$\min_{x,z} I_U(x) + I_V(z) \quad \text{при} \quad x - z = 0$$

Каждый цикл ADMM включает две проекции:

$$x_k = P_U(z_{k-1} - u_{k-1})$$

$$z_k = P_V(x_k + u_{k-1})$$

$$u_k = u_{k-1} + x_k - z_k$$

Коррекция u даёт сходимость там, где простые поочерёдные проекции заклиниваются.

Figure 5: Сложную проекцию на пересечение $U \cap V$ (красный «серп») ADMM заменяет двумя простыми: на круг U и на полуплоскость V по отдельности.

Проекция на пересечение: ADMM в действии

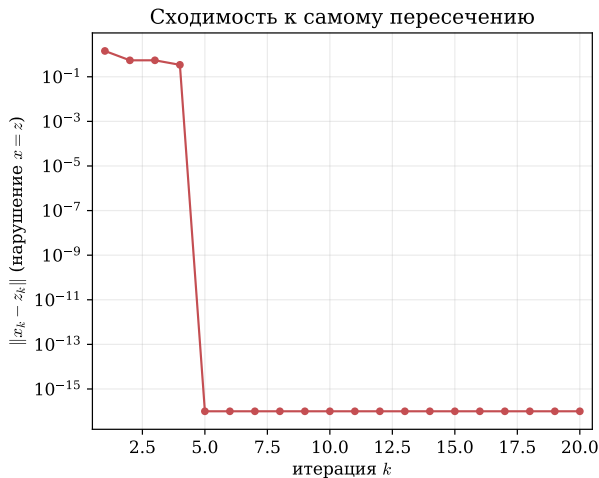
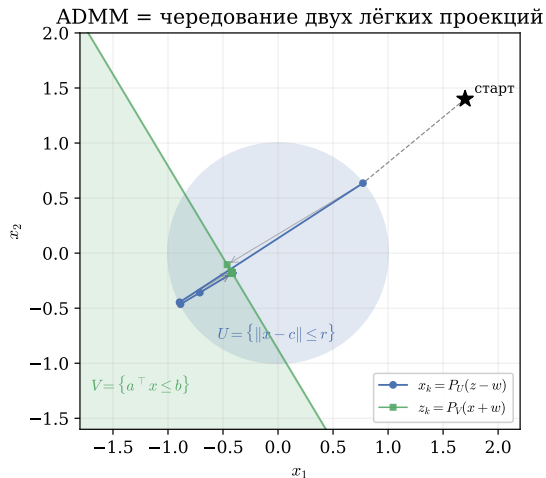


Figure 6: ADMM как чередующиеся проекции на круг U и полуплоскость V . Траектория зигзагом сходится в пересечение, $\|x_k - z_k\| \rightarrow 0$: сложную проекцию заменяем двумя простыми.

LASSO через ADMM: расщепить гладкое и негладкое

LASSO $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$: два слагаемых **разной природы**. Вводим копию $z = x$ и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

LASSO через ADMM: расщепить гладкое и негладкое

LASSO $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$: два слагаемых **разной природы**. Вводим копию $z = x$ и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

Шаги ADMM (масштабированная форма u):

$$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho(z_k - u_k)),$$

$$z_{k+1} = S_{\lambda/\rho}(x_{k+1} + u_k),$$

$$u_{k+1} = u_k + (x_{k+1} - z_{k+1}),$$

где $S_\tau(v) = \text{sign}(v) \cdot \max(|v| - \tau, 0)$ — мягкий порог.

	$\frac{1}{2} \ Ax - b\ ^2$	$\lambda \ z\ _1$
гладкость	да	нет
инструмент	система	мягкий порог
сложность	матрица A	покоординатно

LASSO через ADMM: расщепить гладкое и негладкое

LASSO $\min_x \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$: два слагаемых **разной природы**. Вводим копию $z = x$ и отдаём каждое слагаемое своему методу:

$$\min_{x,z} \frac{1}{2} \|Ax - b\|^2 + \lambda \|z\|_1 \quad \text{при } x = z$$

Шаги ADMM (масштабированная форма u):

	$\frac{1}{2} \ Ax - b\ ^2$	$\lambda \ z\ _1$
гладкость	да	нет
инструмент	система	мягкий порог
сложность	матрица A	покоординатно

$$x_{k+1} = (A^T A + \rho I)^{-1} (A^T b + \rho (z_k - u_k)),$$

$$z_{k+1} = S_{\lambda/\rho}(x_{k+1} + u_k),$$

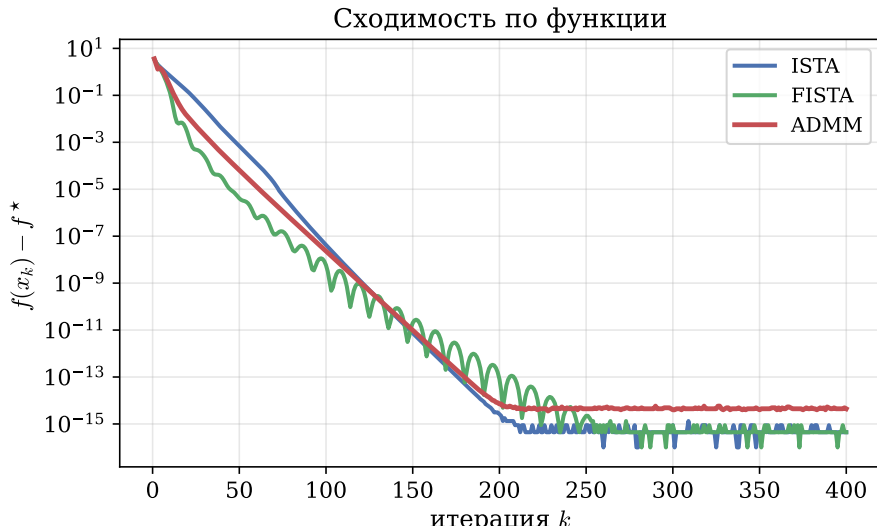
$$u_{k+1} = u_k + (x_{k+1} - z_{k+1}),$$

где $S_\tau(v) = \text{sign}(v) \cdot \max(|v| - \tau, 0)$ — мягкий порог.

x -шаг есть линейная система (естественная для квадратичной части), z -шаг есть мягкий порог в замкнутой форме (естественный для ℓ_1), u -шаг есть цена за рассогласование $x \neq z$. Факторизация $(A^T A + \rho I)$ считается один раз.

LASSO через ADMM: сравнение с ISTA / FISTA

LASSO: $m = 200$, $n = 500$, $\|x^*\|_0 = 25$



Где ADMM явно выигрывает: оператор в регуляризаторе

Возьмём **fused LASSO** $\min_x \frac{1}{2} \|x - y\|^2 + \lambda \|Dx\|_1$, где D — разностная матрица, $(Dx)_i = x_{i+1} - x_i$. У $\lambda \|Dx\|_1$ нет замкнутого прох (разность связывает координаты), поэтому прокс-градиент напрямую неприменим; прямой конкурент — субградиентный метод.

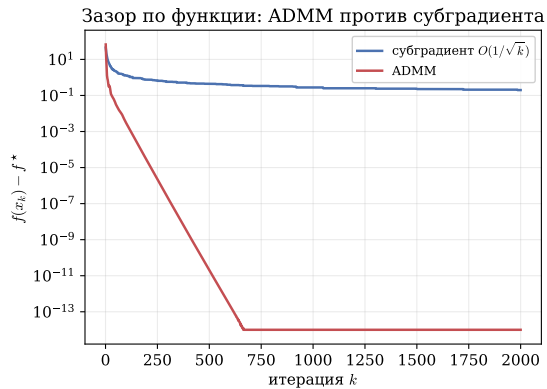
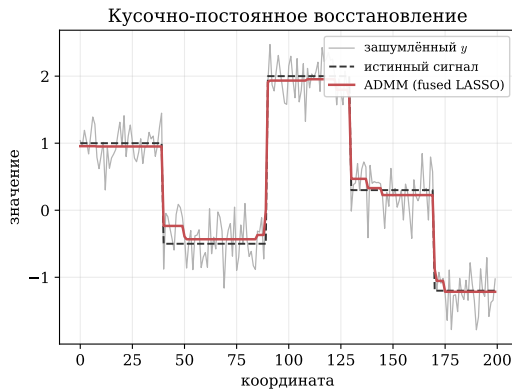


Figure 8: Слева: восстановление кусочно-постоянного сигнала из шума. Справа: ADMM ($z = Dx$) за ~ 700 итераций доходит до машинной точности, субградиент застревает на ~ 0.2 .

Двойственные методы и прежний инструментарий курса

«У меня уже есть прокс-градиент и проекция, зачем ADMM?» У каждого из прежних методов есть структура, на которой он перестаёт работать.

Метод курса	Где перестаёт работать	Что даёт ADMM
Субградиентный	негладкость, медленный $O(1/\sqrt{k})$	использует прокс негладкой части (усадка)
Прокс-градиент / FISTA	прокс g сложен (TV с оператором, ≥ 2 слагаемых)	расщепляет на части с простым прокс
Проекция градиента	проекция только на простые множества	пересечение множеств через расщепление
Франк–Вульф	составные негладкие целевые функции, распределённость	разбивает на блоки, цена связности есть множитель

Двойственные методы и прежний инструментарий курса

«У меня уже есть прокс-градиент и проекция, зачем ADMM?» У каждого из прежних методов есть структура, на которой он перестаёт работать.

Метод курса	Где перестаёт работать	Что даёт ADMM
Субградиентный	негладкость, медленный $O(1/\sqrt{k})$	использует прох негладкой части (усадка)
Прокс-градиент / FISTA	прокс g сложен (TV с оператором, ≥ 2 слагаемых)	расщепляет на части с простым прох
Проекция градиента	проекция только на простые множества	пересечение множеств через расщепление
Франк–Вульф	составные негладкие целевые функции, распределённость	разбивает на блоки, цена связности есть множитель

Прежние методы рассчитаны на **одну** удобную структуру; ADMM расщепляет задачу на части, каждую из которых они решают. На простой гладкой задаче ускоренные методы по-прежнему предпочтительнее.

Эволюция методов курса на LASSO

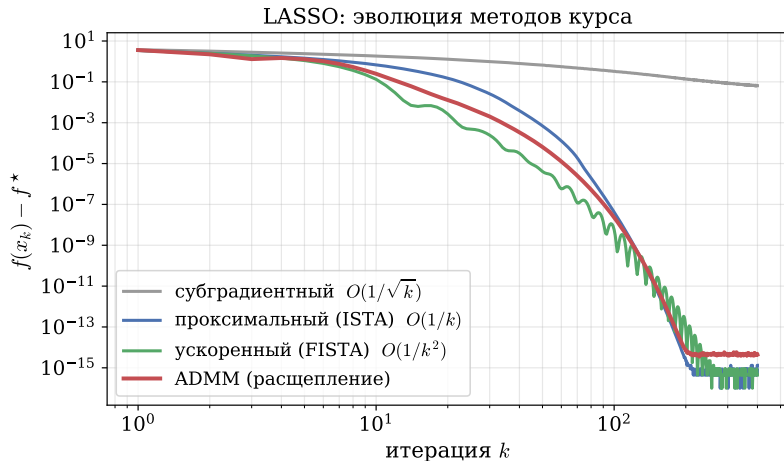


Figure 9: Субградиентный метод сходится медленно ($O(1/\sqrt{k})$) и застревает; проксимальный и ускоренный используют прох негладкой части; ADMM расщепляет задачу. Именно прох и расщепление есть причина перехода от субградиента к этим методам.

Какой метод когда выбирать: итог

Метод	Сильная выпуклость?	Параллельный?	Негладкая часть r ?	Скорость
Двойственный подъём	нужна	нет	через прокс	линейно (при μ -сильной)
Двойственная декомпозиция	нужна	да (B задач)	через прокс	как у двойственного подъёма
ALM	не нужна	нет	да	линейно, устойчив
ADMM	не нужна	да	да (z -шаг)	$O(1/k)$, на практике быстро

Какой метод когда выбирать: итог

Метод	Сильная выпуклость?	Параллельный?	Негладкая часть r ?	Скорость
Двойственный подъём	нужна	нет	через прокс	линейно (при μ -сильной)
Двойственная декомпозиция	нужна	да (B задач)	через прокс	как у двойственного подъёма
ALM	не нужна	нет	да	линейно, устойчив
ADMM	не нужна	да	да (z -шаг)	$O(1/k)$, на практике быстро

- Выбор метода сведён в таблицу выше; на практике чаще всего встречается случай «не сильно выпукло и есть негладкая часть» $\Rightarrow$ **ADMM**.

Какой метод когда выбирать: итог

Метод	Сильная выпуклость?	Параллельный?	Негладкая часть r ?	Скорость
Двойственный подъём	нужна	нет	через прокс	линейно (при μ -сильной)
Двойственная декомпозиция	нужна	да (B задач)	через прокс	как у двойственного подъёма
ALM	не нужна	нет	да	линейно, устойчив
ADMM	не нужна	да	да (z -шаг)	$O(1/k)$, на практике быстро

- Выбор метода сведён в таблицу выше; на практике чаще всего встречается случай «не сильно выпукло и есть негладкая часть» $\Rightarrow$ **ADMM**.
- ADMM применим широко, но требует подбора ρ и точного решения подзадач; на гладкой хорошо обусловленной задаче ускоренные методы быстрее.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**

$$\frac{1}{2}x^\top Px + q^\top x \text{ при } l \leq Ax \leq u.$$

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**
 $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**
 $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**
 $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи**
 $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи** $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

SCS (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи** $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

SCS (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи** $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

SCS (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.
- Поставляется с CVXPY как универсальный конический солвер.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи** $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

SCS (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.
- Поставляется с CVXPY как универсальный конический солвер.

ADMM в реальных солверах: OSQP и SCS

ADMM — не только учебный метод. На нём построены промышленные солверы, незаметно работающие под капотом **CVXPY** (самый популярный на Python язык моделирования выпуклой оптимизации).

OSQP (*Operator Splitting QP; Stellato, Banjac, Goulart, Bemporad, Boyd, 2020*)

- Решает выпуклые **квадратичные задачи** $\frac{1}{2}x^\top Px + q^\top x$ при $l \leq Ax \leq u$.
- Специализированный ADMM: матрица ККТ-системы **постоянна** — факторизация считается **один раз** и переиспользуется на всех итерациях.
- После настройки работает **без делений**, поддерживает **тёплый старт**, генерирует автономный C-код без динамической памяти.
- Отсюда — рабочая лошадка **встраиваемого MPC** (управление в реальном времени, робототехника) и **QP-движок по умолчанию в CVXPY**.

SCS (*Splitting Conic Solver; O'Donoghue, Chu, Parikh, Boyd, 2016*)

- Решает большие выпуклые **конические задачи**: LP, SOCP, **SDP**, экспоненциальный конус.
- ADMM поверх однородного самодвойственного вложения; метод первого порядка, масштабируется на большие размерности.
- Поставляется с CVXPY как универсальный конический солвер.

Итог. Решая QP в CVXPY, вы запускаете ADMM внутри OSQP; коническую задачу или SDP — ADMM внутри SCS. Канонический источник — монография Boyd, Parikh, Chu, Peleato, Eckstein (*Foundations and Trends in ML*, 2011), одна из самых цитируемых работ области.

Непрерывные методы оптимизации

От дискретных шагов к непрерывному времени

Все методы курса до сих пор были **дискретными**: последовательность точек x_k . Посмотрим на оптимизацию под другим углом — как на **непрерывную траекторию** $x(t)$, заданную дифференциальным уравнением.

От дискретных шагов к непрерывному времени

Все методы курса до сих пор были **дискретными**: последовательность точек x_k . Посмотрим на оптимизацию под другим углом — как на **непрерывную траекторию** $x(t)$, заданную дифференциальным уравнением.

Перепишем градиентный спуск как разностную схему:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) \quad \Rightarrow \quad \frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k).$$

От дискретных шагов к непрерывному времени

Все методы курса до сих пор были **дискретными**: последовательность точек x_k . Посмотрим на оптимизацию под другим углом — как на **непрерывную траекторию** $x(t)$, заданную дифференциальным уравнением.

Перепишем градиентный спуск как разностную схему:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) \quad \Rightarrow \quad \frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k).$$

Слева — разностное приближение производной. Устремляя шаг $\alpha \rightarrow 0$, получаем обыкновенное дифференциальное уравнение — **градиентный поток** (gradient flow):

!

$$\frac{dx}{dt} = -\nabla f(x), \quad x(0) = x_0.$$

Зачем непрерывное время

- **Нет шага.** Исчезают вопросы про выбор α , line-search и расписания — анализ проще.

Зачем непрерывное время

- **Нет шага.** Исчезают вопросы про выбор α , line-search и расписания — анализ проще.

Зачем непрерывное время

- **Нет шага.** Исчезают вопросы про выбор α , line-search и расписания — анализ проще.
- **Аналитические решения.** Для квадратичной $f(x) = \frac{1}{2}x^\top Ax$ поток линеен:
 $\dot{x} = -Ax \Rightarrow x(t) = e^{-At}x_0$ — точная формула.

Зачем непрерывное время

- **Нет шага.** Исчезают вопросы про выбор α , line-search и расписания — анализ проще.
- **Аналитические решения.** Для квадратичной $f(x) = \frac{1}{2}x^\top Ax$ поток линеен:
 $\dot{x} = -Ax \Rightarrow x(t) = e^{-At}x_0$ — точная формула.

Зачем непрерывное время

- **Нет шага.** Исчезают вопросы про выбор α , line-search и расписания — анализ проще.
- **Аналитические решения.** Для квадратичной $f(x) = \frac{1}{2}x^\top Ax$ поток линеен:
 $\dot{x} = -Ax \Rightarrow x(t) = e^{-At}x_0$ — точная формула.
- **Один поток — много методов.** Разные дискретизации одного ОДУ дают разные алгоритмы (явный Эйлер $\rightarrow$ GD, неявный $\rightarrow$ проксимальный метод).

Зачем непрерывное время

- **Нет шага.** Исчезают вопросы про выбор α , line-search и расписания — анализ проще.
- **Аналитические решения.** Для квадратичной $f(x) = \frac{1}{2}x^\top Ax$ поток линеен:
 $\dot{x} = -Ax \Rightarrow x(t) = e^{-At}x_0$ — точная формула.
- **Один поток — много методов.** Разные дискретизации одного ОДУ дают разные алгоритмы (явный Эйлер $\rightarrow$ GD, неявный $\rightarrow$ проксимальный метод).

Зачем непрерывное время

- **Нет шага.** Исчезают вопросы про выбор α , line-search и расписания — анализ проще.
- **Аналитические решения.** Для квадратичной $f(x) = \frac{1}{2}x^\top Ax$ поток линеен: $\dot{x} = -Ax \Rightarrow x(t) = e^{-At}x_0$ — точная формула.
- **Один поток — много методов.** Разные дискретизации одного ОДУ дают разные алгоритмы (явный Эйлер $\rightarrow$ GD, неявный $\rightarrow$ проксимальный метод).
- **Чистые доказательства.** Сходимость доказывается через **функции Ляпунова** — короче и прозрачнее дискретных выкладок.

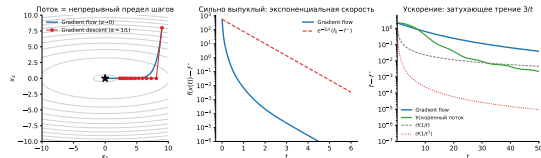


Figure 10: Слева: поток (непрерывный предел) и градиентный спуск (шаги) идут в одну точку. В центре: сильно выпуклый случай — экспоненциальная скорость (фактическая быстрее консервативной оценки $e^{-2\mu t}$). Справа: затухающее трение $3/t$ ускоряет поток.

Дискретизация: явный и неявный Эйлер

Градиентный поток $\frac{dx}{dt} = -\nabla f(x)$ можно дискретизировать по-разному.
Явный Эйлер — градиент в текущей точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

Дискретизация: явный и неявный Эйлер

Градиентный поток $\frac{dx}{dt} = -\nabla f(x)$ можно дискретизировать по-разному.

Явный Эйлер — градиент в текущей точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

$$\boxed{x_{k+1} = x_k - \alpha \nabla f(x_k)} \quad (\text{GD})$$

обычный **градиентный спуск**.

Дискретизация: явный и неявный Эйлер

Градиентный поток $\frac{dx}{dt} = -\nabla f(x)$ можно дискретизировать по-разному.

Явный Эйлер — градиент в текущей точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

(GD)

обычный **градиентный спуск**.

Неявный Эйлер — градиент в новой точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_{k+1})$$

$$\nabla \left[\frac{1}{2\alpha} \|x - x_k\|^2 + f(x) \right] \Big|_{x=x_{k+1}} = 0$$

Дискретизация: явный и неявный Эйлер

Градиентный поток $\frac{dx}{dt} = -\nabla f(x)$ можно дискретизировать по-разному.

Явный Эйлер — градиент в текущей точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

$$\boxed{x_{k+1} = x_k - \alpha \nabla f(x_k)}$$

обычный **градиентный спуск**.

Неявный Эйлер — градиент в новой точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_{k+1})$$

$$\nabla \left[\frac{1}{2\alpha} \|x - x_k\|^2 + f(x) \right] \Big|_{x=x_{k+1}} = 0$$

(GD)

$$\boxed{x_{k+1} = \text{prox}_{\alpha f}(x_k)}$$

(PPM)

проксимальный метод.

Дискретизация: явный и неявный Эйлер

Градиентный поток $\frac{dx}{dt} = -\nabla f(x)$ можно дискретизировать по-разному.

Явный Эйлер — градиент в текущей точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_k)$$

$$\boxed{x_{k+1} = x_k - \alpha \nabla f(x_k)}$$

обычный **градиентный спуск**.

Неявный Эйлер — градиент в новой точке:

$$\frac{x_{k+1} - x_k}{\alpha} = -\nabla f(x_{k+1})$$

$$\nabla \left[\frac{1}{2\alpha} \|x - x_k\|^2 + f(x) \right] \Big|_{x=x_{k+1}} = 0$$

(GD)

$$\boxed{x_{k+1} = \text{prox}_{\alpha f}(x_k)}$$

(PPM)

проксимальный метод.

Одно и то же ОДУ порождает и явный градиентный спуск, и безусловно устойчивый проксимальный метод: выбор схемы дискретизации = выбор алгоритма.

Сходимость: выпуклый случай $\mathcal{O}(1/t)$

Пусть f выпукла и дифференцируема, $f^\star = f(x^\star)$.

Сходимость: выпуклый случай $\mathcal{O}(1/t)$

Пусть f выпукла и дифференцируема, $f^\star = f(x^\star)$.

1. Монотонное убывание. По правилу цепочки

$$\frac{d}{dt}f(x(t)) = \nabla f(x)^\top \dot{x} = -\|\nabla f(x)\|^2 \leq 0,$$

значение f вдоль потока не растёт.

Сходимость: выпуклый случай $\mathcal{O}(1/t)$

Пусть f выпукла и дифференцируема, $f^\star = f(x^\star)$.

1. Монотонное убывание. По правилу цепочки

$$\frac{d}{dt}f(x(t)) = \nabla f(x)^\top \dot{x} = -\|\nabla f(x)\|^2 \leq 0,$$

значение f вдоль потока не растёт.

2. Расстояние до решения. Для $\mathcal{D}(t) = \frac{1}{2}\|x(t) - x^\star\|^2$

$$\dot{\mathcal{D}} = (x - x^\star)^\top \dot{x} = -(x - x^\star)^\top \nabla f(x) \stackrel{\text{выпукл.}}{\leq} -(f(x) - f^\star),$$

так как $f^\star \geq f(x) + \nabla f(x)^\top (x^\star - x)$.

Сходимость: выпуклый случай $\mathcal{O}(1/t)$

Пусть f выпукла и дифференцируема, $f^\star = f(x^\star)$.

1. Монотонное убывание. По правилу цепочки

$$\frac{d}{dt}f(x(t)) = \nabla f(x)^\top \dot{x} = -\|\nabla f(x)\|^2 \leq 0,$$

значение f вдоль потока не растёт.

2. Расстояние до решения. Для $\mathcal{D}(t) = \frac{1}{2}\|x(t) - x^\star\|^2$

$$\dot{\mathcal{D}} = (x - x^\star)^\top \dot{x} = -(x - x^\star)^\top \nabla f(x) \stackrel{\text{выпукл.}}{\leq} -(f(x) - f^\star),$$

так как $f^\star \geq f(x) + \nabla f(x)^\top (x^\star - x)$.

3. Интегрируем от 0 до t и пользуемся монотонностью $f(x(u)) \geq f(x(t))$:

$$t(f(x(t)) - f^\star) \leq \int_0^t (f(x(u)) - f^\star) du \leq -\int_0^t \dot{\mathcal{D}} du = \mathcal{D}(0) - \mathcal{D}(t).$$

Сходимость: выпуклый случай $\mathcal{O}(1/t)$

Пусть f выпукла и дифференцируема, $f^\star = f(x^\star)$.

1. Монотонное убывание. По правилу цепочки

$$\frac{d}{dt}f(x(t)) = \nabla f(x)^\top \dot{x} = -\|\nabla f(x)\|^2 \leq 0,$$

значение f вдоль потока не растёт.

2. Расстояние до решения. Для $\mathcal{D}(t) = \frac{1}{2}\|x(t) - x^\star\|^2$

$$\dot{\mathcal{D}} = (x - x^\star)^\top \dot{x} = -(x - x^\star)^\top \nabla f(x) \stackrel{\text{выпукл.}}{\leq} -(f(x) - f^\star),$$

так как $f^\star \geq f(x) + \nabla f(x)^\top (x^\star - x)$.

3. Интегрируем от 0 до t и пользуемся монотонностью $f(x(u)) \geq f(x(t))$:

$$t(f(x(t)) - f^\star) \leq \int_0^t (f(x(u)) - f^\star) du \leq -\int_0^t \dot{\mathcal{D}} du = \mathcal{D}(0) - \mathcal{D}(t).$$



$$f(x(t)) - f^\star \leq \frac{\|x_0 - x^\star\|^2}{2t} = \mathcal{O}\left(\frac{1}{t}\right).$$

Сходимость: сильно выпуклый случай (экспонента)

Пусть f μ -сильно выпукла. Тогда (монотонность градиента сильно выпуклой функции, $\nabla f(x^*) = 0$):

$$(x - x^*)^\top \nabla f(x) = (x - x^*)^\top (\nabla f(x) - \nabla f(x^*)) \geq \mu \|x - x^*\|^2.$$

Сходимость: сильно выпуклый случай (экспонента)

Пусть f μ -сильно выпукла. Тогда (монотонность градиента сильно выпуклой функции, $\nabla f(x^*) = 0$):

$$(x - x^*)^\top \nabla f(x) = (x - x^*)^\top (\nabla f(x) - \nabla f(x^*)) \geq \mu \|x - x^*\|^2.$$

Сходимость по аргументу. Для $\mathcal{D}(t) = \frac{1}{2} \|x - x^*\|^2$:

$$\dot{\mathcal{D}} = -(x - x^*)^\top \nabla f(x) \leq -\mu \|x - x^*\|^2 = -2\mu \mathcal{D} \Rightarrow \boxed{\|x(t) - x^*\|^2 \leq e^{-2\mu t} \|x_0 - x^*\|^2.}$$

Сходимость: сильно выпуклый случай (экспонента)

Пусть f μ -сильно выпукла. Тогда (монотонность градиента сильно выпуклой функции, $\nabla f(x^*) = 0$):

$$(x - x^*)^\top \nabla f(x) = (x - x^*)^\top (\nabla f(x) - \nabla f(x^*)) \geq \mu \|x - x^*\|^2.$$

Сходимость по аргументу. Для $\mathcal{D}(t) = \frac{1}{2} \|x - x^*\|^2$:

$$\dot{\mathcal{D}} = -(x - x^*)^\top \nabla f(x) \leq -\mu \|x - x^*\|^2 = -2\mu \mathcal{D} \Rightarrow \boxed{\|x(t) - x^*\|^2 \leq e^{-2\mu t} \|x_0 - x^*\|^2.}$$

Сходимость по функции. Сильная выпуклость влечёт условие Поляка–Лоясиевича

$\|\nabla f(x)\|^2 \geq 2\mu(f(x) - f^*)$, поэтому

$$\frac{d}{dt}(f(x) - f^*) = -\|\nabla f(x)\|^2 \leq -2\mu(f(x) - f^*) \Rightarrow \boxed{f(x(t)) - f^* \leq e^{-2\mu t}(f(x_0) - f^*)}.$$

Сходимость: сильно выпуклый случай (экспонента)

Пусть f μ -сильно выпукла. Тогда (монотонность градиента сильно выпуклой функции, $\nabla f(x^*) = 0$):

$$(x - x^*)^\top \nabla f(x) = (x - x^*)^\top (\nabla f(x) - \nabla f(x^*)) \geq \mu \|x - x^*\|^2.$$

Сходимость по аргументу. Для $\mathcal{D}(t) = \frac{1}{2} \|x - x^*\|^2$:

$$\dot{\mathcal{D}} = -(x - x^*)^\top \nabla f(x) \leq -\mu \|x - x^*\|^2 = -2\mu \mathcal{D} \Rightarrow \boxed{\|x(t) - x^*\|^2 \leq e^{-2\mu t} \|x_0 - x^*\|^2.}$$

Сходимость по функции. Сильная выпуклость влечёт условие Поляка–Лоясиевича

$\|\nabla f(x)\|^2 \geq 2\mu(f(x) - f^*)$, поэтому

$$\frac{d}{dt}(f(x) - f^*) = -\|\nabla f(x)\|^2 \leq -2\mu(f(x) - f^*) \Rightarrow \boxed{f(x(t)) - f^* \leq e^{-2\mu t}(f(x_0) - f^*)}.$$

Экспоненциальная скорость — непрерывный аналог линейной сходимости $(1 - 2\mu\alpha)^k$ дискретного градиентного спуска. Для оценки по функции достаточно уже условия PL.

Ускоренный поток (Su–Boyd–Candès)

Вспомним одну из форм ускоренного метода Нестерова:

$$x_{k+1} = y_k - \alpha \nabla f(y_k), \quad y_k = x_k + \frac{k-1}{k+2}(x_k - x_{k-1}).$$

Ускоренный поток (Su–Boyd–Candès)

Вспомним одну из форм ускоренного метода Нестерова:

$$x_{k+1} = y_k - \alpha \nabla f(y_k), \quad y_k = x_k + \frac{k-1}{k+2}(x_k - x_{k-1}).$$

Su, Boyd, Candès (2015) показали: при $\alpha \rightarrow 0$ (время $t = k\sqrt{\alpha}$) это дискретизация ОДУ **второго порядка**

!

$$\ddot{X} + \frac{3}{t}\dot{X} + \nabla f(X) = 0.$$

Ускоренный поток (Su–Boyd–Candès)

Вспомним одну из форм ускоренного метода Нестерова:

$$x_{k+1} = y_k - \alpha \nabla f(y_k), \quad y_k = x_k + \frac{k-1}{k+2}(x_k - x_{k-1}).$$

Su, Boyd, Candès (2015) показали: при $\alpha \rightarrow 0$ (время $t = k\sqrt{\alpha}$) это дискретизация ОДУ **второго порядка**

!

$$\ddot{X} + \frac{3}{t}\dot{X} + \nabla f(X) = 0.$$

Это уравнение **массы в потенциале** f с трением: инерция $\ddot{X}$ даёт разгон, а коэффициент трения $3/t$ **затухает** со временем. Постоянное трение дало бы метод тяжёлого шарика; именно убывающее трение $3/t$ обеспечивает ускорение.

Ускоренный поток: доказательство $\mathcal{O}(1/t^2)$

Возьмём функцию Ляпунова (энергию)

$$E(t) = t^2(f(X(t)) - f^\star) + 2\left\|X(t) - x^\star + \frac{t}{2}\dot{X}(t)\right\|^2.$$

Ускоренный поток: доказательство $\mathcal{O}(1/t^2)$

Возьмём функцию Ляпунова (энергию)

$$E(t) = t^2(f(X(t)) - f^\star) + 2\left\|X(t) - x^\star + \frac{t}{2}\dot{X}(t)\right\|^2.$$

Первое слагаемое: $\frac{d}{dt}[t^2(f(X) - f^\star)] = 2t(f(X) - f^\star) + t^2\nabla f(X)^\top \dot{X}.$

Ускоренный поток: доказательство $\mathcal{O}(1/t^2)$

Возьмём функцию Ляпунова (энергию)

$$E(t) = t^2(f(X(t)) - f^*) + 2\left\|X(t) - x^* + \frac{t}{2}\dot{X}(t)\right\|^2.$$

Первое слагаемое: $\frac{d}{dt}[t^2(f(X) - f^*)] = 2t(f(X) - f^*) + t^2\nabla f(X)^\top \dot{X}$.

Обозначим $w = X - x^* + \frac{t}{2}\dot{X}$. Подставляя $\ddot{X} = -\frac{3}{t}\dot{X} - \nabla f$:

$$\dot{w} = \dot{X} + \frac{1}{2}\dot{X} + \frac{t}{2}\ddot{X} = \frac{3}{2}\dot{X} + \frac{t}{2}\left(-\frac{3}{t}\dot{X} - \nabla f\right) = -\frac{t}{2}\nabla f(X).$$

Ускоренный поток: доказательство $\mathcal{O}(1/t^2)$

Возьмём функцию Ляпунова (энергию)

$$E(t) = t^2(f(X(t)) - f^\star) + 2\left\|X(t) - x^\star + \frac{t}{2}\dot{X}(t)\right\|^2.$$

Первое слагаемое: $\frac{d}{dt}[t^2(f(X) - f^\star)] = 2t(f(X) - f^\star) + t^2\nabla f(X)^\top \dot{X}$.

Обозначим $w = X - x^\star + \frac{t}{2}\dot{X}$. Подставляя $\ddot{X} = -\frac{3}{t}\dot{X} - \nabla f$:

$$\dot{w} = \dot{X} + \frac{1}{2}\dot{X} + \frac{t}{2}\ddot{X} = \frac{3}{2}\dot{X} + \frac{t}{2}\left(-\frac{3}{t}\dot{X} - \nabla f\right) = -\frac{t}{2}\nabla f(X).$$

Поэтому $\frac{d}{dt}[2\|w\|^2] = 4w^\top \dot{w} = -2t(X - x^\star)^\top \nabla f - t^2\dot{X}^\top \nabla f$.

Ускоренный поток: доказательство $\mathcal{O}(1/t^2)$

Возьмём функцию Ляпунова (энергию)

$$E(t) = t^2(f(X(t)) - f^*) + 2\left\|X(t) - x^* + \frac{t}{2}\dot{X}(t)\right\|^2.$$

Первое слагаемое: $\frac{d}{dt}[t^2(f(X) - f^*)] = 2t(f(X) - f^*) + t^2\nabla f(X)^\top \dot{X}$.

Обозначим $w = X - x^* + \frac{t}{2}\dot{X}$. Подставляя $\ddot{X} = -\frac{3}{t}\dot{X} - \nabla f$:

$$\dot{w} = \dot{X} + \frac{1}{2}\dot{X} + \frac{t}{2}\ddot{X} = \frac{3}{2}\dot{X} + \frac{t}{2}\left(-\frac{3}{t}\dot{X} - \nabla f\right) = -\frac{t}{2}\nabla f(X).$$

Поэтому $\frac{d}{dt}[2\|w\|^2] = 4w^\top \dot{w} = -2t(X - x^*)^\top \nabla f - t^2\dot{X}^\top \nabla f$.

Складываем — члены $t^2\nabla f^\top \dot{X}$ сокращаются:

$$\dot{E} = 2t\left[(f(X) - f^*) - \nabla f(X)^\top (X - x^*)\right] \stackrel{\text{выпукл.}}{\leq} 0.$$

Ускоренный поток: доказательство $\mathcal{O}(1/t^2)$

Возьмём функцию Ляпунова (энергию)

$$E(t) = t^2(f(X(t)) - f^*) + 2\left\|X(t) - x^* + \frac{t}{2}\dot{X}(t)\right\|^2.$$

Первое слагаемое: $\frac{d}{dt}[t^2(f(X) - f^*)] = 2t(f(X) - f^*) + t^2\nabla f(X)^\top \dot{X}$.

Обозначим $w = X - x^* + \frac{t}{2}\dot{X}$. Подставляя $\ddot{X} = -\frac{3}{t}\dot{X} - \nabla f$:

$$\dot{w} = \dot{X} + \frac{1}{2}\dot{X} + \frac{t}{2}\ddot{X} = \frac{3}{2}\dot{X} + \frac{t}{2}\left(-\frac{3}{t}\dot{X} - \nabla f\right) = -\frac{t}{2}\nabla f(X).$$

Поэтому $\frac{d}{dt}[2\|w\|^2] = 4w^\top \dot{w} = -2t(X - x^*)^\top \nabla f - t^2\dot{X}^\top \nabla f$.

Складываем — члены $t^2\nabla f^\top \dot{X}$ сокращаются:

$$\dot{E} = 2t\left[(f(X) - f^*) - \nabla f(X)^\top (X - x^*)\right] \stackrel{\text{выпукл.}}{\leq} 0.$$

Значит E не растёт: $E(t) \leq E(0) = 2\|x_0 - x^*\|^2$. Второе слагаемое неотрицательно, поэтому

$$f(X(t)) - f^* \leq \frac{2\|x_0 - x^*\|^2}{t^2} = \mathcal{O}\left(\frac{1}{t^2}\right).$$

Ускоренный поток: доказательство $\mathcal{O}(1/t^2)$

Возьмём функцию Ляпунова (энергию)

$$E(t) = t^2(f(X(t)) - f^*) + 2\left\|X(t) - x^* + \frac{t}{2}\dot{X}(t)\right\|^2.$$

Первое слагаемое: $\frac{d}{dt}[t^2(f(X) - f^*)] = 2t(f(X) - f^*) + t^2\nabla f(X)^\top \dot{X}$.

Обозначим $w = X - x^* + \frac{t}{2}\dot{X}$. Подставляя $\ddot{X} = -\frac{3}{t}\dot{X} - \nabla f$:

$$\dot{w} = \dot{X} + \frac{1}{2}\dot{X} + \frac{t}{2}\ddot{X} = \frac{3}{2}\dot{X} + \frac{t}{2}\left(-\frac{3}{t}\dot{X} - \nabla f\right) = -\frac{t}{2}\nabla f(X).$$

Поэтому $\frac{d}{dt}[2\|w\|^2] = 4w^\top \dot{w} = -2t(X - x^*)^\top \nabla f - t^2\dot{X}^\top \nabla f$.

Складываем — члены $t^2\nabla f^\top \dot{X}$ сокращаются:

$$\dot{E} = 2t\left[(f(X) - f^*) - \nabla f(X)^\top (X - x^*)\right] \stackrel{\text{выпукл.}}{\leq} 0.$$

Значит E не растёт: $E(t) \leq E(0) = 2\|x_0 - x^*\|^2$. Второе слагаемое неотрицательно, поэтому

$$f(X(t)) - f^* \leq \frac{2\|x_0 - x^*\|^2}{t^2} = \mathcal{O}\left(\frac{1}{t^2}\right).$$

Бонус: стохастический поток

Как смоделировать стохастичность (как в SGD) в непрерывном времени? Добавим к потоку шум:

$\frac{dx}{dt} = -\nabla f(x) + \xi$. Аккуратная запись — **стохастическое** дифференциальное уравнение

$$dx(t) = -\nabla f(x(t)) dt + \sigma dW(t),$$

где $W(t)$ — винеровский процесс. Анализировать можно двояко: следить за траекториями $x(t)$ либо за эволюцией плотности распределения $\rho(t)$.

Бонус: стохастический поток

Как смоделировать стохастичность (как в SGD) в непрерывном времени? Добавим к потоку шум:

$\frac{dx}{dt} = -\nabla f(x) + \xi$. Аккуратная запись — **стохастическое** дифференциальное уравнение

$$dx(t) = -\nabla f(x(t)) dt + \sigma dW(t),$$

где $W(t)$ — винеровский процесс. Анализировать можно двояко: следить за траекториями $x(t)$ либо за эволюцией плотности распределения $\rho(t)$.

! Уравнение Фоккера–Планка

$$\frac{\partial \rho}{\partial t} = \nabla \cdot (\rho \nabla f) + \frac{\sigma^2}{2} \Delta \rho.$$

Стационарное распределение $\rho_\infty \propto e^{-2f/\sigma^2}$ концентрируется у минимумов f : шум не даёт сойтись в точку, но «температура» σ^2 управляет разбросом.

Непрерывное и дискретное: словарь

Непрерывный объект	Дискретизация	Метод	Скорость
$\dot{x} = -\nabla f$	явный Эйлер	градиентный спуск	$\mathcal{O}(1/t)$ / $e^{-2\mu t}$
$\dot{x} = -\nabla f$	неявный Эйлер	проксимальный метод	устойчив при любом шаге
$\ddot{X} + \frac{3}{t}\dot{X} + \nabla f = 0$	симплектическая	Нестеров	$\mathcal{O}(1/t^2)$
$dx = -\nabla f dt + \sigma dW$	Эйлер–Маруяма	SGD	стационарное $\rho_\infty \propto e^{-2f/\sigma^2}$

Непрерывное и дискретное: словарь

Непрерывный объект	Дискретизация	Метод	Скорость
$\dot{x} = -\nabla f$	явный Эйлер	градиентный спуск	$\mathcal{O}(1/t) / e^{-2\mu t}$
$\dot{x} = -\nabla f$	неявный Эйлер	проксимальный метод	устойчив при любом шаге
$\ddot{X} + \frac{3}{t}\dot{X} + \nabla f = 0$	симплектическая	Нестеров	$\mathcal{O}(1/t^2)$
$dx = -\nabla f dt + \sigma dW$	Эйлер–Маруяма	SGD	стационарное $\rho_\infty \propto e^{-2f/\sigma^2}$

Главная мысль. За знакомыми методами стоит небольшое семейство потоков; «непрерывный» взгляд даёт короткие доказательства через функции Ляпунова и подсказывает новые методы через выбор схемы дискретизации.

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2015)

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2015)
- Francis Bach — Gradient flows (blog)

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2015)
- Francis Bach — Gradient flows (blog)
- Off Convex Path — Gradient flow и gradient descent

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2015)
- Francis Bach — Gradient flows (blog)
- Off Convex Path — Gradient flow и gradient descent
- OSQP: Stellato, Banjac, Goulart, Bemporad, Boyd, *Math. Prog. Comp.* 12(4), 2020 — osqp.org

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2015)
- Francis Bach — Gradient flows (blog)
- Off Convex Path — Gradient flow и gradient descent
- OSQP: Stellato, Banjac, Goulart, Bemporad, Boyd, *Math. Prog. Comp.* 12(4), 2020 — osqp.org
- SCS: O'Donoghue, Chu, Parikh, Boyd, *JOTA* 169(3), 2016 — cvxgrp.org/scs

- W. Su, S. Boyd, E. Candès — A Differential Equation for Modeling Nesterov's Accelerated Gradient Method (2015)
- Francis Bach — Gradient flows (blog)
- Off Convex Path — Gradient flow и gradient descent
- OSQP: Stellato, Banjac, Goulart, Bemporad, Boyd, *Math. Prog. Comp.* 12(4), 2020 — osqp.org
- SCS: O'Donoghue, Chu, Parikh, Boyd, *JOTA* 169(3), 2016 — cvxgrp.org/scs
- Boyd, Parikh, Chu, Peleato, Eckstein — Distributed Optimization via ADMM (Found. Trends ML, 2011)